Java Date-Time API Reference

Modern, Immutable, and Thread-Safe Date/Time Handling (java.time)

Java 8+	Immutable	ISO-8601
INTRODUCED IN	THREAD SAFETY	DEFAULT STANDARD

1. Core Concepts & Core Architecture

Prior to Java 8, date handling via `java.util.Date` and `Calendar` suffered from mutability, poor design, and thread-safety flaws. The modern `java.time` package provides clear separation between human-readable calendar time and machine-centric timestamp epochs.



2. Primary Classes Overview

Class Name	Description	Example Output
`LocalDate`	Date without time or time-zone	`2026-10-26`
`LocalTime`	Time without date or time-zone	`14:30:45.123`
`LocalDateTime`	Combines date and time without zone	`2026-10-26T14:30:45`
`ZonedDateTime`	Complete date-time with time-zone ID	`2026-10-26T14:30+05:30[Asia/Kolkata]`
`Instant`	Nanosecond timestamp from Unix Epoch	`2026-10-26T09:00:00Z`

3. Working with Local Date & Time

`LocalDate`, `LocalTime`, and `LocalDateTime` are ideal for domain models where time-zone offsets are either irrelevant or implicit (such as birth dates, store operating hours, or flight schedules).

Core Operations Example

```
// Creation & Inspection
LocalDate today = LocalDate.now();
LocalDate specificDate = LocalDate.of(2026, Month.OCTOBER, 26);

// Fluent Immutability (Returns a new instance)
LocalDate nextWeek = today.plusWeeks(1);
LocalDate pastMonth = today.minusMonths(2);

// Extracting Fields
int year = today.getYear();
DayOfWeek dayOfWeek = today.getDayOfWeek();
```

4. Time Zones & Offsets

When dealing with distributed systems, server logs, or global user schedules, handling time zones is crucial. Java distinguishes between exact fixed offsets (`ZoneOffset`) and geographical zone rules (`ZoneId`).

Time Zone Conversion Code Pattern

```
// Capture current time in UTC
Instant nowUtc = Instant.now();

// Convert Instant to specific ZonedDateTime
ZoneId indiaZone = ZoneId.of("Asia/Kolkata");
ZonedDateTime istTime = nowUtc.atZone(indiaZone);

// Convert between zones seamlessly
ZoneId nyZone = ZoneId.of("America/New_York");
ZonedDateTime nyTime = istTime.withZoneSameInstant(nyZone);

System.out.println("IST: " + istTime);
System.out.println("NYC: " + nyTime);
```

ZoneId

Represents a region identifier like `Europe/London`. Handles Daylight Saving Time (DST) changes automatically.

ZoneOffset

Represents a fixed offset from UTC/Greenwich, such as `+05:30`, without dynamic DST transition rules.

5. Period vs. Duration & Temporal Adjusters

Java differentiates between date-based time amounts (`Period`) and time-based amounts (`Duration`).

Measuring Elapsed Time & Adjustments

```
// Period: Date-based (Years, Months, Days)
LocalDate start = LocalDate.of(2024, 1, 1);
LocalDate end = LocalDate.of(2026, 10, 26);
Period period = Period.between(start, end); // 2 Years, 9 Months, 25 Days

// Duration: Time-based (Seconds, Nanoseconds)
Instant t1 = Instant.now();
// ... perform task ...
Instant t2 = Instant.now();
Duration duration = Duration.between(t1, t2);

// TemporalAdjusters: Business logic queries
LocalDate nextFriday = LocalDate.now()
    .with(TemporalAdjusters.next(DayOfWeek.FRIDAY));
```

6. Formatting and Parsing

`DateTimeFormatter` replaces the legacy, non-thread-safe `SimpleDateFormat`.

Pattern Symbol	Meaning	Example Output
`yyyy / MM / dd`	4-digit Year / 2-digit Month / Day	`2026/10/26`
`HH:mm:ss`	24-hour time with minutes and seconds	`14:30:00`
`EEEE, MMM d`	Full Day name, Short Month, Day of month	`Monday, Oct 26`
`ISO_LOCAL_DATE_TIME`	Standard ISO format constant	`2026-10-26T14:30:00`

7. Migration Best Practices

- **Always use UTC** for persistence and database storage. Convert to local time zone only at display boundaries.
- **Interoperability:** Convert legacy `java.util.Date` using `date.toInstant()`.
- **Avoid mutation bugs:** Remember that all `java.time` objects are immutable. Calling `date.plusDays(1)` returns a new object.